



VNiVERSIDAD
D SALAMANCA

Signals and Systems

MATLAB Lab Sessions · 2nd Year Computer Engineering

Álvaro Lozano Murciego and André Sales Mendes

Universidad de Salamanca

1st Edition • 2026

CONTENTS

1	Session 1. Introduction to MATLAB	3
1.1	Coming from C	3
1.2	The MATLAB environment	4
1.3	Exercise 1. Scalar variables	5
1.4	Exercise 2. Vectors and matrices	7
1.5	Exercise 3. The <code>size</code> function	8
1.6	Exercises 4 and 5. Element-wise operations	9
1.7	Exercise 6. Plotting	10
1.8	Exercise 7. The <code>for</code> loop	11
1.9	Exercise 8. Functions in <code>.m</code> files	12
1.10	Reference: elementary signals	13

This book collects the lab material for the **Signals and Systems** course (2nd year of the Computer Engineering degree). The lab sessions use **MATLAB**, a tool designed for numerical and matrix computation that lets you define, plot and transform signals with very few lines of code.

How to use this book

Each lab session has its own chapter. In each one you will find:

- **Learning objectives** at the start, so you know what you should master by the end.
- **Explanations and code examples** ready to copy into MATLAB and modify.
- **Exercise statements**, highlighted in boxes, in the same order as the session handout.
- **Proposed solutions**, collapsed: try to solve each exercise before expanding them.

If you already program in C

The first session is written for students who already know C. Many explanations compare the syntax of both languages directly so you can build on what you already know.

Sessions

- *Session 1. Introduction to MATLAB*

SESSION 1. INTRODUCTION TO MATLAB

This is the reference guide for the first lab session. If you already know how to program in C, you will find that MATLAB has a different syntax but a familiar logic. Use this page together with the exercise handout: each section corresponds to one of the exercises.

Learning objectives

- Recognise the most important syntax differences between C and MATLAB.
- Use the environment: Command Window, Workspace, script editor and current folder.
- Define scalars, complex numbers, vectors and matrices, and access their elements.
- Distinguish matrix operations (`*`, `/`, `^`) from element-wise operations (`.*`, `./`, `.^`).
- Plot signals with `plot` and label the figures properly.
- Write `for` loops and functions in `.m` files.

1.1 Coming from C

MATLAB is not a general-purpose language: it is designed for matrix and numerical computation. Many things that need libraries and loops in C are a single line here. The price is a syntax with several differences that cause errors at first. [Table 1.1](#) summarises the ones you will notice on day one.

Table1.1: Basic equivalences between C and MATLAB.

In C you write...	In MATLAB it is...	Why does it change?
<code>int a = 5;</code>	<code>a = 5;</code>	No type declarations. Everything is double by default.
<code>// comment</code>	<code>% comment</code>	<code>%</code> starts a comment. The remainder of a division is computed with <code>mod(a, b)</code> .
<code>printf("x=%d\n", x);</code>	<code>disp(x)</code> or just <code>x</code>	Without a trailing <code>;</code> , MATLAB prints the result automatically. <code>fprintf</code> , almost identical to <code>printf</code> , also exists.
<code>v[0] = 1;</code>	<code>v(1) = 1;</code>	Indices start at 1, not 0, and are written in parentheses.
<code>a * b</code> (two numbers)	<code>a .* b</code> (two vectors)	<code>*</code> is the matrix product. Element-wise multiplication uses <code>.*</code> .
<code>#include <math.h></code>	(nothing)	There are no includes: <code>sin</code> , <code>cos</code> , <code>exp</code> , <code>sqrt</code> ... are always available.
<code>}</code> (closes a block)	<code>end</code>	<code>for</code> , <code>if</code> , <code>while</code> and function blocks are closed with <code>end</code> .
<code>;</code> (ends a statement)	<code>;</code> (suppresses output)	The <code>;</code> is optional: leave it out when you want to see the value in the console.

The most common day-one mistake

Using `*` instead of `.*` when operating on two vectors. If MATLAB answers `Error using * Incorrect dimensions for matrix multiplication`, that is the problem.

The two snippets below do the same thing: fill a vector with the numbers 1 to 10. Notice the starting index, the `for` header and the `end` that replaces the closing brace.

Listing 1.1: `for` loop in C.

```
int v[10];
for (int k = 0; k < 10; k++) {
    v[k] = k + 1;
}
```

Listing 1.2: The same loop in MATLAB.

```
v = zeros(1, 10);
for k = 1:10
    v(k) = k;
end
```

1.2 The MATLAB environment

When you open MATLAB you will see several work areas:

- **Command Window (console):** type a command, press Enter and see the result immediately.
- **Workspace:** lists the variables in memory, with their size and value.
- **Editor:** where you write scripts and functions in `.m` files.
- **Current Folder:** the current working folder. MATLAB only finds your `.m` files if they are in it (or on the path).

A few commands keep the environment clean and you will use them constantly:

Listing 1.3: Essential clean-up commands.

```
clc          % clear the console (like cls in Windows)
clear        % delete all variables from the Workspace
clear a b    % delete only a and b
who          % list the variables that currently exist
whos         % list variables with their type and size

% Good habit: start every script with
clc; clear;
```

Script or console

You can type directly in the console or save the code in a `.m` file and run it with **F5** (or the *Run* button). In this session use scripts: that way you can fix and rerun your code easily.

1.3 Exercise 1. Scalar variables

Statement

Define variables, with any name, that store the values given in the handout table: numbers, fractions, roots, trigonometric functions, logarithms and complex numbers.

1.3.1 Assignment

Assignment works as in C (`variable = value`), with two differences: there are no types, and the trailing `;` only stops MATLAB from printing the result.

Listing 1.4: Assignment and predefined constants.

```
a = 3;          % silent
b = -5.7;       % silent
b = -5.7        % no ; -> prints "b = -5.7000" in the console

% Predefined constants (do not use them as variable names)
c = pi;         % 3.14159...
d = exp(1);     % number e = 2.71828...
```

1.3.2 Fractions and roots

MATLAB evaluates `*` and `/` from left to right, just like C. That is why `5/2*7` equals $(5/2) \cdot 7 = 17.5$ and not $5/14$. Always use parentheses when a fraction has several factors in the denominator.

Listing 1.5: Fractions, roots and powers.

```
f1 = 5 / (2*7); % 5/14 = 0.3571
f2 = 1/3;       % 0.3333...

r1 = sqrt(2);   % square root of 2 = 1.4142...
```

(continues on next page)

(continued from previous page)

```
r2 = 2^(1/3);      % cube root of 2: ^ is the power (pow in C)
r3 = 8^(2/3);      % = 4
```

1.3.3 Trigonometry

Trigonometric functions **always work in radians**, just like those in `math.h`.

Listing 1.6: Trigonometric functions and their inverses.

```
s1 = sin(pi/4);      % 0.7071
s2 = cos(pi/3);      % 0.5
s3 = tan(pi/4);      % 1

s4 = asin(0.5);      % pi/6
s5 = atan2(1, 1);    % pi/4 (arctangent with the correct quadrant)
```

1.3.4 Logarithms and exponential

Listing 1.7: Logarithms in different bases and exponential.

```
l1 = log(exp(1));    % natural logarithm: ln(e) = 1
l2 = log10(100);     % base-10 logarithm -> 2
l3 = log2(8);        % base-2 logarithm -> 3
e1 = exp(-2);        % e^(-2) = 0.1353
```

1.3.5 Complex numbers

Complex numbers are essential in Signals and Systems. A complex number can be written in rectangular form, $z = a + jb$, or in polar form, $z = r e^{j\theta}$, where $r = |z|$ is the magnitude and θ the argument. In MATLAB the imaginary unit is written `i` or `j`; the safest form is `1i` or `1j`.

Listing 1.8: Defining and operating with complex numbers.

```
z1 = 3 + 4i;         % rectangular form
z2 = 2*exp(1i*pi/3); % polar form: r = 2, theta = pi/3

magnitude = abs(z1); % |z1| = 5
phase      = angle(z1); % argument = atan2(4, 3) = 0.9273 rad
conjugate   = conj(z1); % conjugate: 3 - 4i
real_part   = real(z1); % real part: 3
imag_part   = imag(z1); % imaginary part: 4
```

Do not use `i` or `j` as counters

`i` and `j` represent the imaginary unit. If you use them as loop variables they will no longer equal $\sqrt{-1}$, and your complex computations will give wrong results without any warning. Use `k`, `n` or `m` for indices.

1.4 Exercise 2. Vectors and matrices

Statement

Define the given vectors and matrices. For the last three (very long vectors) it is **mandatory** to use the `:` operator notation; you cannot type every element by hand.

1.4.1 Explicit definition

Elements are written between square brackets. A space (or comma) separates columns and a semicolon separates rows.

Listing 1.9: Row vectors, column vectors and matrices.

```
% Row vector: elements separated by spaces or commas
v = [1 2 3 4 5];
v = [1, 2, 3, 4, 5];           % equivalent

% Column vector: elements separated by semicolons
c = [1; 2; 3; 4; 5];

% 3x3 matrix: rows are separated by ;
A = [1 2 3; 4 5 6; 7 8 9];

% Transpose: the ' operator turns row <-> column
ct = c';                       % column -> row
```

1.4.2 The `:` operator

The `:` (colon) operator generates vectors with a regular pattern. Its syntax is `start:step:end`; if you omit the step, it is 1. The step can be fractional or negative.

Listing 1.10: Vectors generated with the `:` operator and with `linspace`.

```
v1 = 1:10;                     % [1 2 3 4 5 6 7 8 9 10]
v2 = 0:0.5:3;                  % [0 0.5 1 1.5 2 2.5 3]
v3 = 10:-1:1;                  % countdown: [10 9 8 ... 1]
v4 = 0:2:20;                   % even numbers from 0 to 20

% linspace(a, b, N): N equally spaced points between a and b
t = linspace(0, 10, 1000);     % 1000 points in [0, 10]

% In Signals, this is how the time axis is defined
t = 0:0.001:10;                % from 0 to 10 with step 0.001 (10001 points)
```

When to use `:` and when to use `linspace`

Use `start:step:end` when the **step** matters (for example, the sampling period). Use `linspace(a, b, N)` when the **number of points** matters. Both create row vectors.

1.4.3 Accessing elements

Remember that the first element has index 1. The word `end` inside an index means “the last one”.

Listing 1.11: Indexing vectors and matrices.

```
v = [10 20 30 40 50];

v(1)           % first element: 10    (v[0] in C)
v(3)           % third element: 30    (v[2] in C)
v(end)         % last element: 50
v(2:4)         % elements 2, 3 and 4: [20 30 40]
v(end-1:end)   % second-to-last and last: [40 50]

% Matrices: A(row, column)
A(1, 2)        % row 1, column 2
A(2, :)        % the whole row 2
A(:, 3)        % the whole column 3
```

1.5 Exercise 3. The `size` function

Statement

Obtain the number of rows and columns of a matrix and of a row vector with the `size` function, and store those values in two variables.

The `size` function returns the dimensions of a variable. Called as `[m, n] = size(A)` it returns rows and columns at once; called as `size(A, dim)` it returns a single dimension (`dim = 1` for rows, `dim = 2` for columns). For vectors, `length(v)` directly gives the number of elements.

Listing 1.12: Three ways to query dimensions.

```
A = [1 2 3; 4 5 6];           % 2x3 matrix

% Way 1: rows and columns at once (what the exercise asks for)
[rows, cols] = size(A);       % rows = 2, cols = 3

% Way 2: a single dimension
rows = size(A, 1);            % number of rows
cols = size(A, 2);            % number of columns

% Way 3: for vectors, length is more convenient
v = 1:10;
n = length(v);                % n = 10
n = numel(v);                 % total number of elements
```

Check your result

For a row vector such as `v = 1:10`, `size(v)` returns `[1 10]`: one row and ten columns. If transposing it (`v'`) gives `[10 1]`, you have understood the difference between a row vector and a column vector.

1.6 Exercises 4 and 5. Element-wise operations

i Statement

- **Exercise 4.** Multiply two vectors element by element using `.*`.
- **Exercise 5.** Divide element by element with `./` and square each element with `.^`.

This is one of the most important differences from C. In MATLAB, `*` is **matrix** multiplication. To operate on two vectors position by position you add a dot before the operator, as summarised in Table 1.2.

Table 1.2: Element-wise operators and their C equivalent.

Operator	Action	“By hand” equivalent in C
<code>.*</code>	$c(k) = a(k) * b(k)$ for every k	for ($k=0$; $k<n$; $k++$) $c[k] = a[k]*b[k]$;
<code>./</code>	$c(k) = a(k) / b(k)$ for every k	for ($k=0$; $k<n$; $k++$) $c[k] = a[k]/b[k]$;
<code>.^</code>	$c(k) = a(k)^p$ for every k	for ($k=0$; $k<n$; $k++$) $c[k] = \text{pow}(a[k], p)$;
<code>*</code>	Matrix product (a different concept)	Not the same: the number of columns of a must match the number of rows of b .

i Proposed solution for exercises 4 and 5

```
v = [1 2 3 4 5];
z = [2 4 1 3 6];

% Exercise 4: element-wise product
prod_ew = v .* z;           % [2 8 3 12 30]

% Exercise 5a: element-wise division
div_ew = v ./ z;           % [0.5 0.5 3 1.3333 0.8333]

% Exercise 5b: square of each element of v
squared = v .^ 2;          % [1 4 9 16 25]
```

Why does this matter in Signals? Because a signal in MATLAB is a vector with thousands of samples. To compute, for example, the damped sine $y(t) = \sin(2\pi t)e^{-t}$ you must multiply sample by sample:

Listing 1.13: Damped sine: element-wise product of two signals.

```
t = 0:0.001:10;
y = sin(2*pi*t) .* exp(-t);
```

i Careful

`sin(t) * exp(-t)` produces the error `Incorrect dimensions for matrix multiplication, because it tries to multiply two row vectors as matrices`. Always use `.*` when operating on two vectors point by point. Multiplying a vector by a **scalar** (`2*t`) does work with `*`.

1.7 Exercise 6. Plotting

Statement

Plot several functions for $t \in [0, 10]$ (the last one for $t \in [-10, 10]$) using `plot`. Use `figure` to show them in separate windows.

The pattern is always the same:

1. Define the vector `t` with the domain required by the exercise and a small step (0.001 gives good resolution).
2. Compute the signal with element-wise operations on `t`.
3. Open a new window with `figure` and draw with `plot(t, y)`.
4. Add labels. A plot without a title and labelled axes is an incomplete plot.

Listing 1.14: Complete pattern for plotting a signal.

```
% Step 1: time axis
t = 0:0.001:10;

% Step 2: signal
y = sin(2*pi*t);

% Step 3: open a window and draw
figure;           % new window (avoids overwriting the previous plot)
plot(t, y);

% Step 4: labels
xlabel('t (s)');
ylabel('y(t)');
title('Sine signal');
grid on;
```

The following signals are typical examples of the kind of functions the exercise asks for. Adapt them to the specific expressions in your handout.

Listing 1.15: Example signals for exercise 6.

```
t = 0:0.001:10;
t2 = -10:0.001:10;      % domain [-10, 10] for the last function

% Decaying exponential
figure; plot(t, exp(-t));
title('e^{-t}'); xlabel('t'); grid on;

% Damped cosine
figure; plot(t, exp(-0.5*t) .* cos(2*pi*t));
title('e^{-0.5t} cos(2\pi t)'); xlabel('t'); grid on;

% Square wave (sign of a sine)
figure; plot(t, sign(sin(pi*t)));
title('Square wave'); xlabel('t'); grid on;

% Sinc on [-10, 10]: sinc(x) = sin(pi x)/(pi x)
figure; plot(t2, sinc(t2));
title('sinc(t)'); xlabel('t'); grid on;
```

i If sinc is not available in your installation

The `sinc` function belongs to the *Signal Processing Toolbox*. If you do not have it, compute $y = \sin(\pi t^2) / (\pi t^2)$; and fix the point $t = 0$, where the division gives NaN, with $y(t^2 == 0) = 1$;

Some plot options that will save you time:

Listing 1.16: Line styles, several curves and axis limits.

```
% Styles: 'colour' + 'line' + 'marker'
plot(t, y, 'b-');      % blue, solid line
plot(t, y, 'r--');     % red, dashed line
plot(t, y, 'go');      % green, circle at each point

% Several curves in the same figure
figure;
hold on;
plot(t, sin(t));
plot(t, cos(t));
legend('sin(t)', 'cos(t)');
hold off;

% Set the axis limits
xlim([0 10]);
ylim([-1.5 1.5]);
```

i Mathematical notation in titles

In `title`, `xlabel` and `ylabel` you can use basic TeX notation: `'e^{-t}'` shows an exponent, `'\pi'` shows the Greek letter and `'f_0'` shows a subscript.

1.8 Exercise 7. The for loop

i Statement

Using a `for` loop, create a vector made up of the natural numbers from 1 to 10.

MATLAB's `for` is different from C's: it does not control a counter with a condition and an increment, it **iterates over the elements of a vector**. On each pass, the loop variable takes the next value of the vector.

Listing 1.17: General structure of the `for` loop.

```
for variable = vector_of_values
    % block of statements (no braces)
end
```

Some common iteration vectors (only the headers are shown):

Listing 1.18: Examples of loop headers.

```

for k = 1:10           % k = 1, 2, ..., 10
for k = 0:2:20         % k = 0, 2, 4, ..., 20
for k = 10:-1:1        % k = 10, 9, 8, ..., 1
for x = [3 7 -1]       % x = 3, then 7, then -1

```

i Proposed solution for exercise 7

```

v = zeros(1, 10);      % preallocate the vector
for k = 1:10
    v(k) = k;
end
disp(v)                % 1 2 3 4 5 6 7 8 9 10

```

i Preallocation

Before a loop that fills a vector, create it with `zeros(1, n)`. Without preallocation MATLAB resizes the vector on every iteration: it works, but it is much slower for large vectors.

1.9 Exercise 8. Functions in .m files

i Statement

Write a MATLAB function that returns the product of all the elements of a vector passed as an argument.

A MATLAB function is saved in a .m file whose name must be **exactly the same** as the function name. The first line defines the outputs, the name and the inputs: `function output = name(input)`. There is no `return` with a value: the function returns whatever the output variable holds when it reaches `end`.

Compare the C version with the MATLAB version. In MATLAB there is no need to pass the size of the vector, because the function can find it with `length`.

Listing 1.19: Product of the elements of a vector in C.

```

double product(double *v, int n) {
    double r = 1.0;
    for (int k = 0; k < n; k++)
        r *= v[k];
    return r;
}

```

i Proposed solution: file product_elements.m

```

function result = product_elements(v)
% PRODUCT_ELEMENTS Returns the product of all the elements of v.
%
% result = product_elements(v)

```

```
%
% Input:  v      - numeric vector (row or column)
% Output: result - scalar with the accumulated product

result = 1;                                % identity element of the product
for k = 1:length(v)
    result = result * v(k);
end
end
```

Once the file is saved, the function can be called from the console or from any script:

Listing 1.20: Calling the function and checking it with `prod`.

```
p = product_elements([1 2 3 4 5]); % p = 120
p = product_elements(1:10);        % p = 3628800 = 10!

% Check: MATLAB includes the prod function
prod([1 2 3 4 5])                  % = 120
```

MATLAB cannot find my function

The file `product_elements.m` must be in the current folder (*Current Folder*). Check where you are with `pwd` and change folder with `cd` or from MATLAB's address bar. Also check that the file name matches the function name.

A function can return several outputs. They are declared in square brackets and collected the same way:

Listing 1.21: Function with two outputs (file `my_max.m`).

```
function [mx, idx] = my_max(v)
% Returns the maximum of v and its position
[ mx, idx] = max(v);
end
```

Listing 1.22: Calling a function with several outputs.

```
[m, n] = my_max([3 8 2 5]); % m = 8, n = 2
m       = my_max([3 8 2 5]); % only the maximum
```

1.10 Reference: elementary signals

These are the signals that will appear most often throughout the course, ready to copy and adapt. The unit step, the rectangular pulse of width a and the ramp are defined as

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad \Pi_a(t) = \begin{cases} 1, & |t| < a/2 \\ 0, & \text{otherwise} \end{cases} \quad r(t) = t u(t) \quad (1.1)$$

In MATLAB, the expressions in (1.1) are built with comparisons: $(t \geq 0)$ equals 1 where the condition holds and 0 where it does not.

Listing 1.23: Elementary signals built with logical comparisons.

```
t = -5:0.001:5;

% Unit step u(t)
u = (t >= 0);

% Rectangular pulse of width a centred at 0
a = 2;
rect = (t >= -a/2) & (t < a/2);

% Ramp r(t) = t u(t)
ramp = t .* (t >= 0);

% Causal damped sine (typical response of a 2nd-order system)
y_am = exp(-0.5*t) .* sin(2*pi*t) .* (t >= 0);
```

With these signals you can compute properties and apply basic transformations. The energy of a signal, $E = \int |x(t)|^2 dt$, is approximated numerically by summing the squared samples and multiplying by the step Δt .

Listing 1.24: Energy, time shift and time reversal.

```
dt = t(2) - t(1); % sampling step

% Signal energy (numerical integration)
E = sum(abs(y_am).^2) * dt;

% Time shift: y(t - t0)
t0 = 1;
y_del = exp(-0.5*(t-t0)) .* sin(2*pi*(t-t0)) .* (t >= t0);

% Time reversal: y(-t)
y_rev = exp(0.5*t) .* sin(-2*pi*t) .* (t <= 0);
```

Session summary

- In MATLAB there are no type declarations, indices start at 1 and blocks are closed with `end`.
- Vectors are created with `[ ]`, with `start:step:end` or with `linspace`.
- Between two vectors, always use `.*`, `./` and `.^`.
- Every plot needs `figure`, `plot`, a title and labelled axes.
- A function lives in a `.m` file with the same name, in the current folder.